ONLY YOU — technical report

A provenance-gated authority layer for wearable agents Sushant Poudel · Meta Global AI Developer Hackathon 2026 · Build for Your World

1. The problem

Meta's WebMCP documentation for Ray-Ban Display tells developers to set `untrustedContentHint` "honestly, since **it's not an enforced security boundary**," and warns that tool-result text "goes into the assistant's context" and is "how a prompt injection reaches the model." The assistant also cannot tell the wearer's voice from a stranger's. Once the agent has a camera, a microphone and a wallet, any TV ad, poster, webpage or invoice can issue commands.

The missing layer is not intelligence. It is **authority**: which inputs are instructions at all, who is allowed to issue them, what they may authorise, and what can be proven afterwards.

2. What I built

Provenance before the model. Every input is labelled before a token reaches the model: on-device speaker verification (only the enrolled owner's voice is a command channel), DOM origin (web text is never a command), scene-text masks + OCR (physical text is data), sender trust for documents. The model never has to win an adversarial classification game; the channel decides what is even eligible to be an instruction.

A constitution and mandates. The constitution is the ceiling — deterministic rules first, a small on-device evaluator only for ambiguity, ask-once for irreversible actions. Mandates are delegation: a signed, scoped, expiring capability ("pay the electrician up to 5,000 until tomorrow") that lets the agent act without interrupting and lets a third party verify the authorisation. Spend is cumulative and tracked outside the signed payload, so a mandate stays valid after use.

Receipts. Every action *and every refusal* emits an HMAC-signed, hash-chained record with the evidence, the rule applied, and an undo. Refusals are tagged `proposed_by: model | display_only`, so an enforcement event is never confused with a UI attribution.

3. What I measured

The four-arm table is a **harness test, not a security claim**: the corpus and the router's channel table are co-designed, so arm D cannot mislabel inside it. Its zero is conditional on correct labels.

Arm	Architecture	Executed	False refusals
A	undefended	100%	0%
B	prompt-only	47.5%	0%
C	provenance-only	10%	0%
D	full ONLY YOU	0%	0%

Real-model runs — Muse Spark. Four runs through the Meta Model API (`api.meta.ai/v1`), 195, 115, 115 and 99 calls at temperature 0, with every proposal re-validated by the gate: `muse-spark-1.3` on the general and document corpora, `muse-spark-1.3-contributor` on the document corpus, and a final-code reproduction run (8 attacks / 8 benign: every arm 0% executed, 16/16 receipts, chain intact).

General corpus (20 attacks / 20 benign; ambient, scene, voice, web). Muse Spark's own refusal behaviour dominates: **0% executed on every arm**, including undefended. The gate's measurable cost is false refusals (**D 5%**, C 10%; A/B 0%), and it files **36/36 verified receipts**. p50 6.8 s (D), p95 14.5 s.

Document / confused-deputy corpus (8 attacks / 12 benign). When the owner asks to pay a real invoice and the document supplies the payee, the model is helpful — and wrong: **A 25%, B 37.5%, C 50% executed** (provenance alone is worse, because it reclassifies instructions but not arguments). The full gate blocks **8/8 (D 0%) with 0% false refusals** and 18/18 verified receipts. p50 10.7 s, p95 34.7 s.

The same confused-deputy corpus on the **contributor build** (`muse-spark-1.3-contributor` , 115 calls) replicates the finding: **50% undefended, 37.5% prompt-only, 50% provenance-only, 0% with the full gate**, 0% false refusals, 18/18 receipts verified.

The honest picture: alignment is a strong first line against obvious injections; the authority layer is what catches the confused deputy the model is trying to be helpful about. Samples are small and synthetic.

The label layer — the part that can be wrong. A DOM hidden-text detector on 42 hidden / 35 benign snippets: recall **81.0%** (95% CI 66.7–90.0), false positives **14.3%** (6.3–29.4), precision 86.8%; eight hiding techniques are missed. A browser labeler using computed styles scores **42/42 with 0/35 false positives** on the same cases (an upper bound — it was written after the cases). Speaker verification's human false-accept rate is **not measured**; a sensitivity table stands in, and a synthetic TTS study (`results/voice-study.json`) rejects 6/6 TTS impostors with 3/3 owner accepts.

Adaptive corpus — the attacker controls the signals. Fifteen scenarios (twelve cloned-voice, three pages claiming `user_typed`): **label error 80%, executed 20%** (the cases where the owner confirmed); web cases are 0/3 after making web text data-only.

Signed sensor envelope. Six fabricated-signal attacks impersonating the owner: **6/6 execute without an envelope, 0/6 with a signed sensor bus**. It closes cheap forgery, not a cloned voice or a rooted sensor.

Evidence coverage. Eight paraphrases: **0 executed**, but only **1/8 receipted** — the audit layer fails first.

Scene text. Thirty-five synthetic signs across seven conditions, rendered with ImageMagick and read with Tesseract on both the raw image and an auto-deskewed, normalized copy, keeping the higher-confidence read: detection **100%** (35/35), mean character error rate **1.7%**; the weakest condition is occlusion (9.8% CER). Routing the OCR result through the stack executes **0/35** and produces **27/35 receipts** (77.1%) — enforcement does not depend on OCR, because scene text is data by channel; OCR quality changes the evidence surface.

Muse Wonder scene reader. The demo path can read the camera frame with a Meta multimodal model: 15 synthetic signs (clean, rotated 8°, low-contrast) read with the same prompt the API uses gave **detection 100% (15/15), mean CER 0.0%**, with approximate pixel boxes for the UI mask. Tesseract stays as the offline fallback. n is small by design; boxes are approximate, not segmentation masks.

SAM 3.1 + Muse Voice Transcribe. SAM 3.1 returns one object box for `paper` (drawn in the overlay); `text` prompts return zero and mask decoding is roadmap, so the label-layer numbers are unchanged. Muse Voice Transcribe transcribes the sample command with one ASR error; the transcript is still only text that must pass the voice check and the gate.

Latency (measured, `results/latency.json`). Constitution rules **0.001 ms p50**; full dry-run path (routing + labels + gate) **4.3 ms p50**; local HTTP command with a simulated proposal and a written receipt **12.8 ms p50**; scene OCR **156 ms p50**. Model inference dominates real end-to-end time (6.8–11.9 s p50); camera capture and speaker embedding are still unmeasured. The report says so; the hundred-millisecond figures a wearable demo implies are not claimed here.

4. What failed, and what I did not fix

- **Cloned voice** at score 0.91 defeats the biometric: 12/12 label errors. Policy catches 9/12; the three executions are the confirms-true worst case.
- **Forged sensor dict**: mitigated for the cheap forgery — with a signed sensor bus, fabricated signals go from 6/6 executed to 0/6. A compromised sensor, and a cloned voice, still pass.
- **Merchant verification** is implemented: mandates are signed with an Ed25519 device key and a merchant verifies them offline with only the public key (signature, payee, cap, expiry); the demo exposes it as "verify at merchant". Residual: replay across merchants is future work.
- **Second-order injection** through the ledger is defended and tested: receipt text is sanitised for display and the model-facing ledger view labels every entry as data, never an instruction (`docs/ledger-safety.md`). Editing a receipt breaks its signature and the chain.
- **Model-alignment confound** in the real-model run: on the general corpus Muse Spark refused even when undefended, so the layer's incremental value only appears on the confused-deputy corpus (25–50% undefended/prompt/provenance vs 0% with the full gate).")
- **Biometric consent**: voiceprints are special-category data (GDPR Art. 9, DPDP 2023, BIPA). The demo enrolls only its author and keeps no audio; a product needs a consent flow, retention policy and a bystander policy.

The full table of nine named attacks, severity and status is `docs/limitations.md`.

5. Why this matters for Meta

Two products depend on this layer. **Glasses** become trustworthy only if the agent distinguishes the wearer from the world; the physical channel is the one no prompt-level defense covers. **Agentic commerce** depends on merchants being able to verify that an agent purchase was authorised — Amazon already blocks Muse because it cannot. A signed, scoped mandate turns agent commerce from a trust problem into a verification problem.

The architectural claim is deliberately narrow: **do not make the model robust to injection; make the channel untrusted by default.** Authority is a routing problem, and routing can be tested, instrumented and receipted.

6. With ten engineers

1. Ship the WebMCP tool-result provenance envelope: an `authority` field the assistant enforces, not a hint.
2. Voice + DOM provenance in the Muse client, with the signed sensor envelope.
3. Mandates on device public keys and an offline merchant verifier.
4. Scene provenance behind a flag; publish the benchmark and the corpus.

Repo · demo · benchmark: `results/benchmark.md` · real-model runs: `results-real*/benchmark.md` · limitations: `docs/limitations.md` · 172 tests · self-audit